Hiérarchie Epic / Feature / Story et documentation du cycle en V : ce qui peut remplacer quoi

Note méthodologique — septembre 2026

---

1. Objet et contexte

Cette note complète la note *Organiser le travail en quatre niveaux : Epic, Feature, Story, Tâche*. Elle en reprend les rôles (experts métier dans leur rôle de Product Owner, Scrum Master, équipes de développement et de modélisation, parties prenantes) et l'exemple fil rouge du circuit de refroidissement à l'eau de mer. Les termes propres au cycle en V sont définis en section 2 ; un glossaire complet figure en section 10.

La question. Un backlog organisé en quatre niveaux porte le besoin métier (epic), sa déclinaison en capacités vérifiables (features), le travail de l'équipe (stories, tâches) et des critères d'acceptation. Le cycle en V (V-model) porte, lui, une chaîne documentaire héritée de la norme américaine MIL-STD-498 : spécification système (SSS), exigences logicielles (SRS), interfaces (IRS), conception (SDD), plan de test (STP), description des tests (STD), rapport de test (STR), et des liens de traçabilité (traceability) entre ces niveaux, jusqu'aux tests qui valident chaque exigence.

La question est de savoir lesquels de ces documents la hiérarchie peut remplacer, et à quelles conditions.

Réponse courte.

- **Ce qui ne se remplace pas** : la SSS, référentiel contractuel du client au niveau système ; la stratégie de test (indépendance, environnements, règles de succès) ; l'accord du client sur la forme des livrables.
- **Ce qui peut être porté par l'outil** : le contenu de la SRS, de la STD, de la STR et de la matrice de traçabilité — à condition que les features soient rédigées comme des exigences vérifiables reliées à la SSS, que les tests soient des objets persistants reliés aux exigences, et que les documents soient générés depuis l'outil plutôt que rédigés à la main.
- **Ce que la norme permet** : MIL-STD-498 définit l'information attendue, pas sa forme ; ses livrables peuvent être le contenu d'un outil, dans un format convenu avec l'acquéreur.

---

2. Rôles et vocabulaire

Rôles. Les rôles sont ceux de la note principale. Dans le schéma étudié ici :

Rôle	Responsabilité dans la chaîne documentaire
Experts métier — Product Owner	Détiennent la SSS côté projet ; décident quelles features réalisent quelle exigence système ; acceptent les stories et valident les features
Scrum Master	Veille à la structure : chaque feature reliée à une exigence, chaque test relié, aucun ticket orphelin ; produit les exports documentaires aux jalons
Équipes de développement et de modélisation	Rédigent les critères d'acceptation des features et des stories, les cas de test, les exécutent et enregistrent les résultats
Parties prenantes et client	Conviennent de la forme des livrables (export de l'outil ou document) et assistent aux revues

Termes indispensables

Terme (EN)	Traduction	Sens dans ce document
SSS — System/Subsystem Specification	Spécification système	Exigences du système, écrites par ou pour le client ; base de la conception et des tests de qualification
SRS — Software Requirements Specification	Spécification des exigences logicielles	Exigences d'un composant logiciel (CSCI) et, pour chacune, la méthode qui prouvera qu'elle est satisfaite
STD — Software Test Description	Description des tests	Cas de test, procédures, résultats attendus, lien vers les exigences
STR — Software Test Report	Rapport de test	Résultats d'exécution, anomalies, environnement, verdict
RTM — Requirements Traceability Matrix	Matrice de traçabilité	Tableau exigences ↔ éléments qui les réalisent ↔ tests qui les vérifient
Baseline	Référence figée	Version identifiée et gelée d'un jeu d'exigences ou d'une livraison
DID — Data Item Description	Description de donnée livrable	Dans MIL-STD-498, la fiche qui définit le contenu attendu d'un livrable
CDRL — Contract Data Requirements List	Liste des données contractuelles	Liste des livrables documentaires exigés par le contrat

---

3. Ce que le cycle en V assure réellement

Les documents du cycle en V sont des supports ; ce qui compte, ce sont les fonctions qu'ils assurent.

Fonction	Document classique	Ce que dit la norme d'origine
Fixer les exigences système	SSS	La DID SSS de MIL-STD-498 spécifie les exigences et, pour chacune, la méthode qui prouvera qu'elle est satisfaite ; elle sert de base à la conception et aux tests de qualification
Dériver les exigences logicielles	SRS	La DID SRS reprend exactement la même logique au niveau du composant logiciel : exigences + méthode de vérification, base de la conception et de la qualification
Tracer dans les deux sens	Matrices de traçabilité	Exigée indépendamment du cycle de vie (voir DO-178C ci-dessous)
Prouver la vérification	STP, STD, STR	Preuves objectives, rejouables
Livrer au client	CDRL	Dépend du contrat

Trois points souvent oubliés :

- **MIL-STD-498 n'impose pas le cycle en V.** Son avant-propos annonce une compatibilité améliorée avec les développements incrémentaux et évolutifs et plus de souplesse dans la préparation des documents (texte de la norme, DTIC). Son annexe G décrit trois stratégies, dont l'évolutive, où les exigences ne sont que partiellement définies au départ puis affinées à chaque build.
- **MIL-STD-498 définit l'information, pas le document.** La norme demande de « définir et enregistrer » l'information ; ses DID précisent le contenu requis quelle que soit la forme qu'il prend, et son vocabulaire a été choisi pour admettre le contenu d'outils d'ingénierie comme livrable (présentation des changements 2167A → 498). Les livrables peuvent prendre des formes variées, du code source aux rapports électroniques, la partie contractante précisant les formats acceptés (MIL-STD-498, Wikipedia).
- **Les normes de sûreté exigent la traçabilité, pas un cycle de vie.** DO-178C ne recommande aucun processus de développement particulier ; agile, DevSecOps ou CI/CD sont admis (Parasoft). En revanche, quel que soit le flux de travail choisi, une traçabilité bidirectionnelle forte doit être établie entre exigences de tous niveaux, code source et données de test (arXiv 2209.13975). Une méthode agile s'y intègre en la formalisant dans les plans (PSAC, SDP) et en traitant explicitement traçabilité, indépendance de la vérification et maîtrise des changements (arXiv 2311.04344).

---

4. Document par document : ce qui peut remplacer quoi

Document du cycle en V	Fonction	Remplaçable ?	Par quoi, dans la hiérarchie	Condition
SSS — exigences système	Fixer le besoin contractuel, toutes disciplines	**Non**	—	Reste le référentiel d'entrée ; ses exigences sont représentées ou reliées dans l'outil
SRS — exigences logicielles	Décliner le besoin en comportements logiciels vérifiables, avec leur méthode de vérification	**Oui, sous conditions**	Features (critères d'acceptation), stories, exigences non fonctionnelles (NFR), chacune reliée à une exigence SSS	Critères rédigés comme des exigences vérifiables ; features conservées et versionnées ; export au format SRS si le contrat l'exige
IRS — interfaces	Décrire les interfaces entre composants ou systèmes	**Partiellement**	Contrat d'interface vivant (schéma de données, spécification d'API) versionné avec le code	Une interface partagée avec une autre équipe ou un autre système reste un document de référence
SDD — conception	Décrire la conception du composant	**Partiellement**	Décisions d'architecture enregistrées (ADR), schémas dans le dépôt, enabler stories	Hors périmètre de cette note
STP — plan de test	Stratégie : environnements, indépendance, règles de succès, régression	**Partiellement**	L'inventaire des tests vient de l'outil ; la stratégie reste un document court	
STD — description des tests	Cas de test, procédures, résultats attendus, lien aux exigences	**Oui, sous conditions**	Cas de test, objets persistants reliés aux stories, aux features et aux exigences SSS	Objets « Test », pas sous-tâches ; exportables

Document du cycle en V	Fonction	Remplaçable ?	Par quoi, dans la hiérarchie	Condition
STR — rapport de test	Résultats, anomalies, environnement, verdict	**Oui, sous conditions**	Exécutions de test (test runs) sur une version identifiée, anomalies liées, rapport de couverture	Version ou build identifiée ; résultats gelés
Matrice de traçabilité	Exigences ↔ réalisation ↔ tests	**Oui**	Rapport de couverture et de traçabilité généré depuis les liens	Liens systématiques, vérifiés par la Definition of Done
SVD — description de version	Contenu d'une livraison	**Oui**	Release de l'outil : features, stories et anomalies livrées	

Lecture du tableau : le document rédigé à la main peut disparaître pour la SRS, la STD, la STR, la matrice et la SVD ; l'information qu'ils portent ne disparaît pas, elle vit dans l'outil et s'exporte. La SSS, la stratégie de test et les interfaces partagées restent des documents.

---

5. Scénario étudié : SSS conservée, SRS / STD / STR portées par l'outil

Principe. La SSS reste l'unique référentiel d'exigences écrit à la main. Tout le reste — déclinaison logicielle, réalisation, tests, résultats — est enregistré dans l'outil au fil des sprints, relié à la SSS, puis exporté quand un document est exigé.

```
SSS (client, référentiel)
 └─ Exigence SYS-042 ──────────────┐
                                   ├─ Feature F1  ← critères d'acceptation
= exigences logicielles
                                   │    ├─ Story US-1 ── Test T-11, T-12 ──
Exécutions (release 3.2) ── Anomalies
                                   │    ├─ Story US-2 ── Test T-13
                                   │    ├─ Story US-3 ── Test T-14
                                   │    └─ Story VAL-1 — Test T-20
(scénario complet)
                                   └─ Feature F4 ...
```

```
Rapport de couverture : SYS-042 → F1, F4 → tests → résultats
   ⇒ contenu de la SRS (features + critères), de la STD (tests), de la STR
(exécutions), de la RTM (liens)
```

Étape 1 — La SSS comme ancre

- Chaque exigence de la SSS existe dans l'outil, soit importée comme ticket de type « Exigence » (lecture seule pour l'équipe, versionné par révision de la SSS), soit reliée depuis un référentiel externe par lien bidirectionnel (intégration Jira – DOORS Next).
- Chaque feature porte un lien « réalise » vers au moins une exigence SSS ; une exigence SSS est couverte par au moins une feature, ou explicitement marquée hors périmètre logiciel.
- Les epics regroupent les exigences d'une même capacité opérationnelle ; elles ne portent pas la traçabilité fine, qui se fait au niveau feature.

Exemple : *SYS-042 — Le simulateur doit permettre à l'opérateur de démarrer et d'arrêter chaque pompe de circulation du circuit eau de mer et d'en observer le débit.* → Feature F1 « Conduire les pompes de circulation ».

Étape 2 — La feature comme unité d'exigence logicielle

La DID SRS demande, pour chaque exigence logicielle, l'énoncé et la méthode qui prouvera qu'elle est satisfaite. Une feature porte exactement ce couple : ses critères d'acceptation sont les exigences, les tests qui leur sont reliés sont la méthode. SAFe présente d'ailleurs son modèle Epic / Feature / Story / NFR comme remplaçant largement les spécifications système et logicielles traditionnelles (SAFe Requirements Model).

Deux conditions rendent cette substitution valable :

- **Les critères sont rédigés comme des exigences**, pas comme des slogans. La norme ISO/IEC/IEEE 29148 attend d'une exigence qu'elle soit notamment nécessaire, non ambiguë, complète, singulière, faisable et vérifiable (caractéristiques 29148, arXiv 2408.10886). « Débit animé cohérent avec l'état des pompes » devient : *Lorsqu'une pompe est en marche, la vue affiche le débit publié par le modèle avec un écart nul ; lorsqu'elle est à l'arrêt, le débit affiché est nul.*
- **La feature persiste.** Une story se ferme quand elle est réalisée ; SAFe rappelle que les stories ne sont pas des exigences (User Story Primer). L'unité d'exigence logicielle est donc la feature, jamais la story : une feature terminée n'est ni supprimée ni réécrite ; toute modification de ses critères passe par une demande de changement tracée, et un champ « composant » permet de régénérer une SRS par composant logiciel (CSCI).

Les exigences non fonctionnelles (performance, déterminisme, sécurité) et les exigences dérivées de la conception, qui ne remontent à aucune exigence SSS, sont portées par des NFR au niveau epic ou feature et par des enabler stories, elles aussi reliées à la SSS ou marquées « dérivées ».

Étape 3 — Stories et tâches : la trace du travail, pas la spécification

Tout le développement est effectivement écrit : stories, tâches, revues, déploiements. Cette trace sert au suivi et à l'audit du travail ; elle ne constitue pas la spécification. Ce qui doit survivre à la fermeture d'une story, c'est le critère de la feature qu'elle réalise et le test qui le prouve.

Étape 4 — Les cas de test comme STD

- Un cas de test est un **objet persistant** de l'outil (type « Test » d'une extension de gestion de tests), relié à la story qu'il vérifie **et** à l'exigence SSS concernée, directement ou via la feature. Une sous-tâche « écrire les tests » n'est pas un cas de test : elle se ferme et disparaît du dossier de preuve.
- Les critères Given / When / Then deviennent des tests automatisés : la même spécification sert de spécification exécutable, de test et de documentation du comportement réel (Cucumber).
- Le scénario complet d'une feature (story de validation VAL-1) est le cas de test de qualification de la feature. SAFe range ce travail de vérification et de conformité parmi les enabler stories (SAFe — Story).
- L'ensemble des cas de test, avec procédures, résultats attendus et liens, contient l'information de la STD.

Étape 5 — Les exécutions comme STR

- Chaque exécution (test run) est enregistrée sur une **version identifiée** (release ou build), avec résultat, environnement et anomalies liées ; les résultats d'une version livrée ne sont plus modifiés.
- Les extensions de test de Jira relient exigence → test → exécution → anomalie et produisent un rapport de traçabilité (TestRail, TestFLO), exportable en PDF pour ce dernier.
- L'ensemble des exécutions d'une version contient l'information de la STR ; le rapport de couverture des exigences contient celle de la matrice de traçabilité.

Étape 6 — Ce que la Sprint Review apporte, et ce qu'elle ne remplace pas

L'acceptation d'une feature par les experts métier en Sprint Review est une **décision** ; le rapport de test est une **preuve**. Les deux sont nécessaires : la revue sans exécutions enregistrées ne vaut pas STR, et des exécutions sans acceptation métier ne valent pas validation. En SAFe, la démonstration intégrée des features est la mesure objective de l'avancement (System Demo) ; le Scrum Guide n'admet en revue que ce qui respecte la Definition of Done, laquelle inclut ici l'exécution enregistrée des tests.

Récapitulatif du scénario

Contenu attendu par la norme	Où il vit dans le scénario	Document généré
Exigences logicielles + méthode de vérification	Features (critères d'acceptation) reliées à la SSS, NFR, enabler stories	SRS
Cas de test, procédures, résultats attendus	Objets « Test » reliés aux stories, features et exigences	STD
Résultats, anomalies, environnement, verdict	Exécutions par version, anomalies liées	STR
Exigences ↔ réalisation ↔ tests	Liens SSS → feature → story → test → exécution	RTM
Contenu livré	Release de l'outil	SVD

---

6. Ce qui reste hors de l'outil

- **La SSS elle-même.** Référentiel du client, au niveau système, couvrant matériel, modèles et logiciel : elle se consomme, elle ne se génère pas depuis un backlog logiciel.
- **La stratégie de test.** Environnements, indépendance de la vérification, règles de succès, politique de régression : un document court, stable, qui fixe les règles que l'outil appliquera. La validation d'un modèle physique contre des données de référence — vérification, validation et accréditation (VV&A) dans le vocabulaire de la simulation — reste une campagne avec rapport, dont les résultats peuvent être attachés aux exécutions de test.
- **Les interfaces partagées.** Une interface utilisée par une autre équipe ou un autre système reste un document de référence, versionné avec le code.
- **La recette et la qualification formelles.** Les preuves viennent de l'outil ; l'événement contractuel demeure.
- **L'accord du client.** MIL-STD-498 laisse à l'acquéreur le choix des formats acceptés ; sans accord explicite sur des livrables générés, les documents restent dus sous leur forme classique.

---

7. Conditions et pièges

Conditions

1. **Un objet « Exigence » distinct de la story** : exigences SSS importées ou reliées, features conservées et versionnées, aucune suppression.
2. **Un lien systématique** : chaque feature vers au moins une exigence SSS ; chaque test vers une story et vers une exigence ; chaque exécution vers une version.

3. **Des critères d'acceptation de qualité exigence** : non ambigus, singuliers, vérifiables (ISO/IEC/IEEE 29148).
4. **Des tests persistants**, rejoués en régression après la fermeture des stories ; le rapport de couverture remplace alors la matrice.
5. **Une Definition of Done qui inclut la conformité** : lien d'exigence présent, test relié et exécuté, trace à jour. C'est le principe « construire la conformité incrémentalement » du white paper Compliance de SAFe.
6. **Une maîtrise des changements** : toute modification d'exigence ou de critère passe par une demande tracée ; une baseline est exportée à chaque livraison.
7. **Une règle d'indépendance** si le contrat l'exige : les tests de validation d'une feature sont écrits ou exécutés par une personne autre que le développeur de la story.
8. **Une génération documentaire** : SRS, STD, STR et matrice exportés depuis l'outil aux jalons, dans un format convenu avec le client.
9. **L'accord du client** sur cette forme de livrables. L'exemple du DoD (section 8) montre que cet accord relève de l'acquéreur, pas de l'équipe.

Pièges

Piège	Conséquence
Traiter les sous-tâches « tests » comme des preuves de test	Les sous-tâches se ferment ; aucun cas de test ni résultat ne subsiste
Écrire les features comme des hypothèses de bénéfice sans critères précis	La SRS générée n'est pas une spécification
Supprimer ou réécrire des features terminées	Perte de la baseline des exigences logicielles
Considérer la Sprint Review comme seule preuve	Décision sans enregistrement d'exécution : pas de STR
Stories reliées à plusieurs exigences sans lien explicite	Couverture impossible à calculer
Exécutions non rattachées à une version	Impossible de dire ce qui a été testé sur la livraison

---

8. Ce qui est documenté ailleurs

Défense américaine. Le Software Acquisition Pathway (DoDI 5000.87) soustrait les programmes logiciels au processus d'exigences traditionnel (JCIDS) et le remplace par un énoncé de besoin capacitaire (CNS), un accord utilisateur (UA) et des backlogs programme priorisés, organisés en capacités et features (Capability Needs Statement, DAU). La stratégie de test, incluant vérification, validation et accréditation, reste obligatoire. C'est le

cas de substitution le plus avancé trouvé ; il porte sur l'expression du besoin, pas sur la vérification.

Audit fédéral américain. Le GAO assimile les exigences aux user stories et retient comme bonnes pratiques l'élicitation, la priorisation et la preuve que le travail réalisé répond aux exigences (GAO Agile Assessment Guide). Il note que les organisations restent habituées à des revues centrées sur les documents décrivant exigences et conception plutôt que sur l'implémentation (GAO-20-590G).

Secteurs réglementés (aéronautique, automobile, médical). Scaled Agile traite le sujet par un système qualité allégé (Lean QMS), la construction incrémentale de la conformité et une vérification/validation continue, avec un méta-modèle d'exigences (white paper Compliance, SAFe).

Outillage. Le schéma courant : les ingénieurs système maintiennent les exigences dans un outil dédié (DOORS, DOORS Next, Polarion), les équipes logicielles travaillent dans Jira, et des liens bidirectionnels relient exigences et epics/stories ; la documentation demandée par les audits est produite depuis ces liens (intégration Jira – DOORS Next). Dans les chaînes DO-178C, des tests pilotés par intégration continue sont réimportés dans l'outil d'exigences pour créer des exécutions traçables (Aerospace Systems, 2026).

---

9. Conclusion

La hiérarchie Epic / Feature / Story / Tâche remplace la partie *organisation du travail* du cycle en V : phases séquentielles, spécifications complètes en amont, matrices tenues à la main. Dans le scénario étudié, la SSS reste le seul document d'exigences écrit à la main ; le contenu de la SRS vit dans les features et leurs critères, celui de la STD dans les cas de test, celui de la STR dans les exécutions par version, celui de la matrice dans le rapport de couverture. Ces documents ne sont plus rédigés : ils sont générés.

Ce scénario ne supprime ni le besoin d'exigences stables, ni la traçabilité bidirectionnelle, ni les preuves de qualification, ni les livrables contractuels : il change l'endroit où ces éléments vivent et la manière dont ils sont produits. Le Manifeste agile lui-même reconnaît une valeur à la documentation ; il la place seulement derrière le logiciel qui fonctionne (agilemanifesto.org).

En une phrase : **remplacer les documents, oui ; remplacer les fonctions qu'ils assurent, non — et la SRS, la STD et la STR sont précisément les documents dont la fonction peut être portée par l'outil.**

---

10. Glossaire

Documents du cycle en V (MIL-STD-498)

Sigle	Nom (EN)	Traduction	Rôle
SSS	System/Subsystem Specification	Spécification système	Exigences du système et méthode de vérification de chacune
SRS	Software Requirements Specification	Spécification des exigences logicielles	Exigences d'un composant logiciel et méthode de vérification de chacune
IRS	Interface Requirements Specification	Spécification des interfaces	Exigences d'interface entre composants ou systèmes
SDD	Software Design Description	Description de conception	Conception d'un composant logiciel
STP	Software Test Plan	Plan de test	Stratégie, environnements, responsabilités de test
STD	Software Test Description	Description des tests	Cas de test, procédures, résultats attendus
STR	Software Test Report	Rapport de test	Résultats d'exécution, anomalies, verdict
SVD	Software Version Description	Description de version	Contenu exact d'une livraison
CSCI	Computer Software Configuration Item	Composant logiciel géré en configuration	Unité à laquelle s'applique une SRS
DID	Data Item Description	Description de donnée livrable	Fiche définissant le contenu attendu d'un livrable
CDRL	Contract Data Requirements List	Liste des données contractuelles	Livrables documentaires exigés par le contrat

Traçabilité et vérification

Terme (EN)	Traduction	Définition courte
Traceability	Traçabilité	Capacité à suivre une exigence vers ce qui la réalise et ce qui la vérifie, et inversement
RTM — Requirements Traceability Matrix	Matrice de traçabilité	Tableau des liens exigences ↔ réalisation ↔ tests
Baseline	Référence figée	Version identifiée et gelée d'un jeu d'exigences ou d'une livraison

Terme (EN)	Traduction	Définition courte
Test case	Cas de test	Objet persistant : conditions, étapes, résultat attendu, lien aux exigences
Test run	Exécution de test	Enregistrement d'une exécution : version, environnement, résultat, anomalies
Requirement coverage	Couverture des exigences	Part des exigences reliées à au moins un test, et état de ces tests
Verification / Validation	Vérification / validation	Prouver que le produit fait ce qui est spécifié / qu'il répond au besoin réel
VV&A	Vérification, validation et accréditation	Démarche propre aux simulations : le modèle est-il conforme, fidèle, et accepté pour l'usage prévu
Qualification testing	Tests de qualification	Campagne formelle prouvant qu'un composant satisfait ses exigences
Acceptance (recette)	Recette	Acceptation formelle par le client
Independence	Indépendance	Séparation entre celui qui réalise et celui qui vérifie
Derived requirement	Exigence dérivée	Exigence issue de la conception, sans parent dans la spécification supérieure
NFR — Non-functional requirement	Exigence non fonctionnelle	Performance, déterminisme, sécurité, ergonomie…

Rôles et niveaux du backlog : voir la note *Organiser le travail en quatre niveaux*, sections 2, 3 et 12.

---

11. Sources

Normes et référentiels

- MIL-STD-498, texte de la norme (DTIC) : https://apps.dtic.mil/sti/pdfs/ADA291266.pdf
- MIL-STD-498, annexe G — Stratégies de programme : https://simonjwright.users.sourceforge.net/pushface.org/mil_498/app-g.htm
- MIL-STD-498, DID System/Subsystem Specification : https://cw.fel.cvut.cz/b181/_media/courses/a4m33sep/materialy/architecture_and_design/mil-std-498_syssubsysspecification.doc
- MIL-STD-498, DID Software Requirements Specification (DI-IPSC-81433) : https://www.cs.uah.edu/~mdb0013/Common/SoftwareEng/SRS.html
- MIL-STD-498, changements par rapport à DOD-STD-2167A (forme des livrables, outils) : https://user.ceng.metu.edu.tr/~gtumuklu/web/SE548/Reading%20Material/498/498_CHN

G.PDF
- MIL-STD-498, présentation générale : https://en.wikipedia.org/wiki/MIL-STD-498
- ISO/IEC/IEEE 29148 — caractéristiques d'une exigence (synthèse, arXiv 2408.10886) : https://arxiv.org/pdf/2408.10886
- DO-178C — What is DO-178C (Parasoft) : https://www.parasoft.com/learning-center/do-178c/what-is/
- DO-178C — Traceability regardless of workflow (arXiv 2209.13975) : https://arxiv.org/pdf/2209.13975
- DO-178C — Evaluating Agile feasibility (arXiv 2311.04344) : https://arxiv.org/pdf/2311.04344
- Scrum Guide 2020 — Definition of Done et Sprint Review : https://scrumguides.org/scrum-guide.html
- Manifeste agile : https://agilemanifesto.org/

Défense et audit

- DoDI 5000.87 — Operation of the Software Acquisition Pathway : https://www.esd.whs.mil/Portals/54/Documents/DD/issuances/dodi/500087p.PDF
- DAU — Capability Needs Statement : https://aaf.dau.edu/aaf/software/cns/
- GAO — Agile Assessment Guide (révision 2023) : https://www.gao.gov/products/gao-24-105506
- GAO — Agile Assessment Guide, GAO-20-590G (texte intégral) : https://www.gao.gov/assets/720/710147.pdf

SAFe

- SAFe Requirements Model (SAFe 4.5 Reference Guide) : https://www.oreilly.com/library/view/safe-45-reference/9780134892917/ch112.xhtml
- SAFe — Story (user stories et enabler stories, dont conformité) : https://framework.scaledagile.com/story
- SAFe — System Demo : https://framework.scaledagile.com/system-demo
- User Story Primer (Scaled Agile) : https://framework.scaledagile.com/wp-content/uploads/2021/07/User-Story-Primer.pdf
- White paper — Achieving Regulatory and Industry Standards Compliance with SAFe : https://framework.scaledagile.com/wp-content/uploads/delightful-downloads/2017/10/White_Paper_Compliance_04-08-17-1.pdf

Outillage et pratiques

- Intégration Jira – IBM DOORS Next (OSLC Connect) : https://www.sodiuswillert.com/en/jira-doors-next-integration
- TestRail — Traceability and test coverage in Jira : https://blog.gurock.com/rjira-traceability-test-coverage/

- TestFLO — Requirement Traceability Report : https://deviniti.com/support/addon/server/testflo-814/latest/requirement-traceability-report/
- Cucumber — Executable specifications and living documentation : https://cucumber.io/docs/guides/overview
- DO-178C software development lifecycle with ARP4754A verification (Aerospace Systems, 2026) : https://link.springer.com/article/10.1007/s42401-026-00455-4